

WEB SCRAPING AND AUTOMATION PIPELINE

OVERVIEW OF THE MARIMO NOTEBOOKS

- `Wk06-07_1acceptNstoreCookies.py`
 - bypasses **bot detection** by a target website
 - handles any **cookie consent banners** that appear (the pop-ups asking you to accept or reject cookies)
 - saves the resulting **cookies** and **local storage** so that later scripts can reuse them
 - collects the **list of URLs** found on the website's landing page
- May be executed with the **terminal command**:
 - `python3 Wk06-07_1acceptNstoreCookies.py`

1a. AVOID BOT DETECTION

- Define settings
 - website to visit, topics to filter URLs by
- Launch a browser (**Playwright** / Chromium)
 - CONTEXT 1 — no evasion:
 - Open a page with no user agent → gets blocked by the website
 - Take screenshot → [screenshot_BotDetected.png](#)
(confirms the block is real)
 - CONTEXT 2 — evasion:
 - Set a realistic user agent string and use tricks to hide automation flags
as though the browser was used by a **human**
 - Open a fresh page → website now loads correctly

1b. ACCEPT & STORE COOKIES

- ■ CONTEXT 2 — evasion (cont'd):
 - **cookie banner visible** → [screenshot_CookiesPopup.png](#)
 - Click the “Accept All” button on the cookie banner
 - **banner gone** → [screenshot_CookiesAccepted.png](#)
 - Extract all URLs from the **landing page** → [urls_raw.csv](#)
 - **Filter URLs by topic keywords** → [urls_filtered.csv](#)
 - Save browser cookies & local storage
→ [cookies.json](#), [localStorage.json](#)

OVERVIEW OF THE MARIMO NOTEBOOKS (CONT'D)

- `Wk06-07_2collect_urls.py`
 - crawls a website starting from a set of **seed URLs**
 - follows links up to a configurable **depth**
 - collects all URLs matching specified **keywords** (e.g., sustainability, ESG)
- May be executed with the **terminal command**:
 - `python3 Wk06-07_2collect_urls.py`

2a. WEBCRAWLING TO COLLECT URLS

- Define settings
 - website to crawl
 - only URLs sharing the **highest two-level domain** will be visited (e.g., ['siemens', 'com'])
 - keywords to look for, **how deep** to follow links
 - **maximum run time** to avoid webcrawling indefinitely
- Read candidate URLs from `urls_filtered.csv` (previously saved)
 - Remove URLs with **redundant sub-paths** → `seedURLs.csv`
 - e.g., `/sustainability/reports/2023` is nested inside `/sustainability/`
 - Load saved browser **cookies** and **local storage** (so the crawler looks like a real user)

2b. WEBCRAWLING TO COLLECT URLS

- Launch a browser (**Playwright** / Chromium)
 - For each **seed URL**:
 - Visit the page
 - **Collect all links** on that page
 - For each link that matches a keyword and is **valid for visit** (e.g., not a [mailto:](#) or [pdf](#))
 - Visit that page and repeat **recursively** up to the **allowed depth**
- Save all URLs discovered → [allURLs.csv](#)
(and [allURLs_df.csv](#) with Filename and Extension columns)
 - Save PDF links only → [pdfURLs.csv](#)
 - Save PDFs matching topics → [pdfscreenedURLs.csv](#)
 - Save pages actually visited → [visitedURLs.csv](#)
 - Will not be visited again in future runs of the crawler

OVERVIEW OF THE MARIMO NOTEBOOKS (CONT'D)

- `Wk06-07_3DLnExtract_OCR.py` automates a PDF data collection pipeline:
 1. **Download** — fetch PDF sustainability reports from a list of screened URLs
 2. **Analyse** — count occurrences of user-defined keywords on each page of the PDFs
 3. **Extract** — save pages with the keywords as individual single-page PDFs

3a. DOWNLOAD AND EXTRACT RELEVANT PAGES

- Define settings
 - **year** to target, **keywords** to search for in PDFs
 - fraction of reports to download (for demo purpose)
- Create output folders
 - `docArchive/DLdocs/` (for downloaded PDFs)
 - `docArchive/pagesExtracted/` (for extracted pages)
- Read the screened PDF URL list from `pdfscreenedURLs.csv` (previously saved)
 - Keep only the URLs containing the target year (e.g. `2019`)
 - Load saved browser **cookies** (so the downloader looks like a real user)

3b. DOWNLOAD AND EXTRACT RELEVANT PAGES

- For each URL in the filtered list (up to the chosen fraction):
 - Check the **download ledger** `df_DL.csv` → Skip if already downloaded
 - Otherwise, download and save the PDF to `DLdocs/`
 - **Update** the download ledger `df_DL.csv` accordingly
 - Check internal **page numbering** of the downloaded PDF
 - if broken → fix it after backing up the original to `bad/` subfolder
 - Extract text from the PDF based on PDF type:
 - if **text-searchable** → use `PyMuPDF`
 - if scanned → use **OCR** tools to recognize text
 - **Count occurrences** of the keywords page-by-page
 - Record and save a list of `[page number, keyword count]` pairs in `df_DL.csv`
- For each page on the list of `[page number, keyword count]` pairs:
 - Extract and save the page to `pagesExtracted/`